

Edge cases (an unhandled routing value) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — trigger it and read what happens

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool | None
    message: str

def node_decide(state):
    return {}

def node_a(state):
    return {"message": "path A"}

def node_b(state):
    return {"message": "path B"}

def route(state):
    if state["flag"] is None:
        return "path_c" # deliberately not a key in the mapping below
    return "path_a" if state["flag"] else "path_b"
```

```

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges("node_decide", route, {"path_a": "node_a", "path_b": "node_b"})
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

graph = builder.compile()
print("compiled fine")

print(graph.invoke({"flag": True, "message": ""}))

try:
    graph.invoke({"flag": None, "message": ""})
except Exception as e:
    print(f"{type(e).__name__}: {e}")

```

Expected output:

```

compiled fine
{'flag': True, 'message': 'path A'}
KeyError: 'path_c'

```

Two things to notice: `builder.compile()` never raises anything — `LangGraph` doesn't check that a routing function's every possible return value is covered by the mapping at compile time, because it can't know every value a Python function might return without running it. The normal case (`flag=True`) still works exactly as before. Only the actual bad case — `flag=None` , which makes `route` return `"path_c"` — raises, and it raises loudly with a `KeyError` naming the unmapped value, not a silent wrong path.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — confirming exactly when it fails, with type hints

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool | None
    message: str

def node_decide(state: GraphState) -> dict:
    return {}

def node_a(state: GraphState) -> dict:
    return {"message": "path A"}

def node_b(state: GraphState) -> dict:
    return {"message": "path B"}

def route(state: GraphState) -> str:
    if state["flag"] is None:
        return "path_c"
    return "path_a" if state["flag"] else "path_b"

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges("node_decide", route, {"path_a": "node_a", "path_b": "node_b"})
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

# compiling never runs route() at all -- it only registers the graph structure
```

```
graph = builder.compile()

test_cases = [True, False, None]
for flag_value in test_cases:
    try:
        result = graph.invoke({"flag": flag_value, "message": ""})
        print(f"flag={flag_value}: OK -> {result['message']}")
    except KeyError as e:
        print(f"flag={flag_value}: FAILED LOUDLY -> KeyError({e})")
```

Expected output:

```
flag=True: OK -> path A
flag=False: OK -> path B
flag=None: FAILED LOUDLY -> KeyError('path_c')
```

Difference from Basic: same finding, with the check run across all 3 input cases in a loop instead of one manual `try / except`, and the `KeyError` caught specifically (instead of the broad `Exception`) now that Basic already confirmed that's the actual type raised — narrowing the `except` clause to the real exception type is the more correct pattern once you know what you're actually catching.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a self-checking routing function with a clear custom message

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

VALID_PATHS = {"path_a", "path_b"}

class GraphState(TypedDict):
    flag: bool | None
    message: str

def node_decide(state: GraphState) -> dict:
    return {}

def node_a(state: GraphState) -> dict:
    return {"message": "path A"}

def node_b(state: GraphState) -> dict:
    return {"message": "path B"}

def route(state: GraphState) -> str:
    if state["flag"] is None:
        result = "path_c"
    else:
        result = "path_a" if state["flag"] else "path_b"

    if result not in VALID_PATHS:
        raise ValueError(
            f"route() returned {result!r} for state {state!r}, "
            f"which isn't one of the handled paths: {VALID_PATHS}"
        )
    return result
```

```

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges("node_decide", route, {"path_a": "node_a", "path_b": "node_b"})
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

graph = builder.compile()

try:
    graph.invoke({"flag": None, "message": ""})
except ValueError as e:
    print(f"ValueError: {e}")

```

Expected output:

```
ValueError: route() returned 'path_c' for state {'flag': None, 'message': ''}, wh
```

Compare this to Intermediate's `KeyError: 'path_c'` — both fail immediately, at the same point in execution. This one's message names the actual routing function, shows the full state that caused the problem, and lists every path that *would* have worked — everything you'd want to know to fix it, in one line, instead of a bare `KeyError` you'd have to trace back to `route()` yourself.

Approach 2 — a fallback path instead of a hard failure, used deliberately

Sometimes failing loudly isn't actually what you want in production — you might prefer routing anything unexpected to a dedicated "unhandled" node that logs the problem and gives a safe response, rather than crashing the whole run. This is a real design choice, not a shortcut — it should be a decision you make on purpose, not something that happens because you forgot to handle a case.

```

def route_with_fallback(state: GraphState) -> str:
    if state["flag"] is None:
        return "unhandled"
    return "path_a" if state["flag"] else "path_b"

def unhandled_node(state: GraphState) -> dict:
    return {"message": f"Unhandled routing case for state: {state}"}

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)

```

```

builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_node("unhandled", unhandled_node)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide",
    route_with_fallback,
    {"path_a": "node_a", "path_b": "node_b", "unhandled": "unhandled"},
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)
builder.add_edge("unhandled", END)

graph = builder.compile()
result = graph.invoke({"flag": None, "message": ""})
print(result["message"])

```

Expected output:

```
Unhandled routing case for state: {'flag': None, 'message': ''}
```

Notice this isn't "silently doing something unexpected" the way the exercise warns against — `"unhandled"` is a real, named node in the mapping, chosen on purpose, that visibly records what happened. That's different from a routing value falling through to some default by accident.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate confirms the failure happens and reads the library's own `KeyError`. Approach 1 keeps the same fail-loudly behavior but gives it a message written for a human, by validating inside `route()` before returning. Approach 2 changes the actual behavior — instead of failing, unexpected values route to a real, visible `"unhandled"` node — which is a legitimate choice for production code, as long as it's a deliberate mapping entry, not a missing one.

Which one should you actually write? During development, let it fail loudly — Intermediate's default `KeyError`, or better, Approach 1's clearer `ValueError`, so a routing bug gets caught immediately while you're building, not buried. Approach 2's fallback node is worth adding once a graph is heading to production and you've decided, on purpose, that an unexpected state value should degrade safely (log it, give a generic response) instead of taking down the whole run — but that's a decision to make deliberately, with a real node behind it, never a default you fall into by not handling a case.